Sistema de questionário — VISE-MT

Documento do instrumento de triagem como ele funciona hoje no backend PHP (`backend-php/`) e no app Flutter (`mobile/`). O aluno responde no celular ou no navegador, inclusive sem internet. A equipe monta o instrumento, libera uma sala e lê indicadores e relatórios. O aluno nunca recebe pontuação nem classificação.

Contrato HTTP resumido: `docs/openapi.yaml`. Esquema: `backend-php/database/schema.sql`.

1. O que o sistema faz

Uma **escola** é o tenant. Um **questionário** pertence a uma escola e a um pesquisador. Ele é organizado em categorias (e, se quiser, subcategorias), perguntas e opções com pontuação. **Regras de classificação** transformam a soma de cada categoria em um rótulo e em um nível de atenção.

O questionário só chega ao aluno por meio de uma **aplicação** (sala): escola inteira, uma turma ou um aluno, com código de 6 caracteres e link `/?sala=CODIGO`.

Ao concluir, o servidor grava um **resultado**: totais por categoria e a classificação correspondente. Painel e Excel agregam esses resultados. O app do aluno mostra só o progresso e uma tela de agradecimento.

```
Escola
 └── Questionário (rascunho → publicado, com versões)
      ├── Categoria (cor, mensagem do avatar, imagem)
      │    ├── Subcategoria (opcional, só organização)
      │    └── Pergunta (múltipla escolha ou texto, peso, obrigatória)
      │         └── Opção (texto, pontos, ordem, emoji, cor)
      └── Regras de classificação (faixa de pontos → rótulo + nível)
           │
           └── Aplicação / sala (alvo, código, janela, status)
                ├── Resposta (uma por aluno + pergunta)
                └── Resultado (totais_json + classificacao_json)
```

2. Papéis

Papéis em `backend-php/src/Roles.php`. `write` inclui criar, editar e excluir. Admin geral (`users.is_superuser`) ignora o escopo e vê todas as escolas.

Recurso	Admin da escola	Pesquisador	Professor
Questionário	leitura	escrita	—
Categoria, subcategoria, pergunta, opção, regra	—	escrita	—
Aplicação (sala)	escrita	escrita	leitura
Resposta e resultado	leitura	leitura	leitura
Avatar	escrita	leitura	—
Termo de aceite	leitura	—	—

O pesquisador, nas escolas em que tem vínculo, enxerga os **próprios** questionários e os marcados como **compartilhados na escola**. Listagens incluem os compartilhados. Edição, exclusão e ações (publicar, nova versão) ficam só nos questionários de que ele é dono (`Scope.php`, parâmetro `$strict`).

O professor vê aplicações da escola inteira, das turmas ligadas a ele (`professor_turma`) ou de alunos dessas turmas. No app, ele só cria sala para as próprias turmas, nunca para a escola toda.

O admin da escola libera salas e lê resultados, mas não monta o instrumento.

3. Modelo de dados

MySQL 5.7+ / MariaDB 10.3+, `utf8mb4`. Quase todas as tabelas do instrumento têm `created_at`, `updated_at` e `deleted_at`. O painel **apaga de verdade** (`DELETE`). As colunas `deleted_at` existem e a API do aluno ignora linhas com `deleted_at` preenchido (categorias, perguntas e opções). Respostas e resultados não têm `deleted_at`.

Datas gravadas em UTC. Painel e relatórios exibem no fuso `APP_TIMEZONE` (padrão `America/Cuiaba`).

3.1 `questionarios`

Coluna	Tipo	Regra
`id`	INT PK	
`escola_id`	INT FK `escolas`	tenant
`pesquisador_id`	INT FK `users`	dono; no formulário, em branco vira o usuário logado
`nome`	VARCHAR(255)	obrigatório
`descricao`	TEXT	
`status`	VARCHAR(30)	`rascunho` (padrão) ou `publicado`
`publico_alvo`	VARCHAR(255)	texto livre (ex.: "9º ano"); não filtra quem pode responder
`versao`	INT	começa em 1
`parent_id`	INT FK `questionarios`	versão anterior; nulo na primeira
`compartilhado_na_escola`	TINYINT	1 = outros pesquisadores da escola podem ver e liberar em sala
`deleted_at`	DATETIME	

Índices em `escola_id`, `pesquisador_id`, `parent_id`.

Trocar a escola do questionário propaga `escola_id` para categorias, subcategorias, perguntas, opções e regras daquele instrumento (`QuestionarioService::propagarEscola`).

3.2 `categorias`

Coluna	Tipo	Regra
`questionario_id`	INT FK	
`escola_id`	INT FK	derivada do questionário
`nome`	VARCHAR(255)	
`ordem`	INT	ordenação no app e nos relatórios
`cor`	VARCHAR(30)	hex, ex. `#14B8A6`. A API envia; o app pinta os cartões com uma paleta própria, por índice
`mensagem_avatar`	TEXT	a API envia e o app guarda no cache local; a tela de introdução usa a mensagem do avatar da escola (`GET /avatar`), não este campo
`imagem_apoio`	VARCHAR(500)	URL. No Android/iOS o app baixa para disco se a URL começa com `http`

3.3 `subcategorias`

Agrupamento opcional dentro da categoria: `categoria_id`, `escola_id`, `nome`, `ordem`. A pergunta pode apontar para uma subcategoria da mesma categoria. O app do aluno **não agrupa** a tela por subcategoria; a API só devolve `subcategoria_id` na pergunta.

3.4 `perguntas`

Coluna	Tipo	Regra
`categoria_id`	INT FK	obrigatória
`subcategoria_id`	INT FK	opcional; tem de ser da mesma categoria
`escola_id`	INT FK	derivada da categoria
`tipo`	VARCHAR(50)	`multipla_escolha` ou `texto`
`texto`	TEXT	enunciado
`ordem`	INT	em branco na criação = fim da categoria (`MAX(ordem)+1`)
`obrigatoria`	TINYINT	padrão 1. O servidor só exige as obrigatórias na finalização
`peso`	DECIMAL(8,2)	padrão `1.00`. Multiplica a pontuação da opção
`imagem`	VARCHAR(500)	URL; o app baixa como a imagem da categoria

O aluno não recebe `peso`.

3.5 `opcoes_resposta`

Coluna	Tipo	Regra
`pergunta_id`	INT FK	
`escola_id`	INT FK	derivada da pergunta
`descricao`	VARCHAR(500)	
`pontuacao`	INT	pode ser negativa. **Nunca vai para o aluno**
`ordem`	INT	
`cor`	VARCHAR(30)	a API envia; o app não usa na UI
`emoji`	VARCHAR(50)	o app mostra junto do texto

Na criação da pergunta de múltipla escolha, as opções podem ser digitadas numa caixa, uma por linha (`Resources::opcoesDoTexto`):

- `Nunca = 0`
- `Às vezes; 1` ou `Às vezes | 1` ou `Às vezes : 1`
- sufixo opcional `pt`, `pts` ou `pontos`
- linha só com texto: a pontuação é a posição, começando em 0

Padrão do formulário:

```
Nunca = 0
Às vezes = 1
Frequentemente = 2
Sempre = 3
```

Múltipla escolha nova exige pelo menos duas opções. Depois dá para editar cada opção no cadastro **Opções de resposta**.

3.6 `regras_classificacao`

Coluna	Tipo	Regra
`escola_id`	INT FK	derivada da categoria ou do questionário
`categoria_id`	INT FK	**é esta coluna que o cálculo usa**
`questionario_id`	INT FK	pode ser preenchida; o cálculo de resultado **não** classifica pelo questionário inteiro
`min_score`	DECIMAL(10,2)	inclusivo
`max_score`	DECIMAL(10,2)	inclusivo; não pode ser menor que `min_score`
`rotulo`	VARCHAR(255)	texto livre (ex.: "Tranquilo", "Sinais de sofrimento")
`descricao`	TEXT	interpretação; vai para o JSON do resultado, não para o aluno
`nivel`	VARCHAR(20)	`baixo`, `moderado` ou `alto`. Vazio = "Sem nível" no painel

Faixas podem se sobrepor. O cálculo pega a primeira regra da categoria com `min_score <= total <= max_score`, ordenada por `id`. Se nenhuma faixa cobre o total, a classificação daquela categoria fica `null`.

3.7 `aplicacoes_questionario`

A sala. Sem ela, o aluno não vê o questionário.

Coluna	Tipo	Regra
`questionario_id`	INT FK	tem de ser da mesma escola
`escola_id`	INT FK	
`alvo_tipo`	VARCHAR(50)	`escola`, `turma` ou `aluno`
`turma_id`	INT FK	obrigatória se o alvo é turma; tem de ser da escola
`aluno_id`	INT FK	obrigatório se o alvo é aluno; tem de ser da escola
`inicia_em` / `termina_em`	DATETIME	gravados e exibidos. A API do aluno **não** bloqueia fora dessa janela; o que vale é `status`
`status`	VARCHAR(30)	`ativa` (padrão) ou `encerrada`
`codigo_sala`	VARCHAR(12) UNIQUE	6 caracteres `A–Z` e `0–9`, gerado se vier vazio

Sala `ativa` sem nenhuma pergunta (categorias e perguntas não excluídas logicamente) é recusada, no painel e no app da equipe.

O app da equipe cria alvo `escola` ou `turma`. Alvo `aluno` existe no painel web.

3.8 `respostas`

Uma linha por `(aplicacao_id, aluno_id, pergunta_id)` — índice único.

Coluna	Uso
`opcao_id`	nulo em texto livre; tem de pertencer à pergunta
`texto`	até 10.000 caracteres

Coluna	Uso
`tempo_gasto_ms`	opcional; o app atual não envia
`dispositivo`	até 255 caracteres; o app atual não envia
`responded_at`	instante em que o aluno respondeu, normalizado para UTC `Y-m-d H:i:s`
`client_uuid`	UUID gerado no aparelho; não é chave única

Se a mesma pergunta chegar de novo, fica a resposta com `responded_at` mais recente. Uma sincronização atrasada não apaga uma resposta mais nova. A comparação é lexicográfica depois da normalização UTC, então o formato é estável.

Pergunta que não pertence ao questionário da sala → 404. Opção de outra pergunta → 422.

3.9 `resultados`

Uma linha por `(aplicacao_id, aluno_id)`.

- `totais_json`: objeto `{ "<categoria_id>": <float> }`. Chaves são strings.
- `classificacao_json`: objeto `{ "<categoria_id>": { "rotulo", "descricao", "nivel" } | null }`.

Recalcular (nova finalização) atualiza a mesma linha. `created_at` da primeira gravação é o que o painel usa como data da conclusão (o seed de demonstração reescreve essa data para espalhar o gráfico).

3.10 Tabelas ao redor do instrumento

- **`avatars`**: `escola_id` nulo = avatar global. A API do aluno escolhe o da escola; se não houver, o global (`ORDER BY escola primeiro`).
- **`termos_aceite`**: um registro por `(aluno_id, versao)`. Guarda `texto_hash` (SHA-256, 64 hex), IP, user-agent e `accepted_at`. O app envia versão `1.0` e um hash fixo. Se o POST falhar (offline), o aceite fica só no aparelho (`termo_aceito_<alunoId>`).
- **`legado_map`**: `chave` → `novo_id`, para a importação do sistema antigo não duplicar.

4. Montar o instrumento

Cadastros no painel `/admin` e na API da equipe `/api/v1/staff/cadastros/{recurso}`. Recursos: `questionarios`, `categorias`, `subcategorias`, `perguntas`, `opcoes`, `regras`, `aplicacoes`.

No detalhe de um registro, o bloco **Relacionados** lista o que aponta para ele (categorias do questionário, perguntas da categoria, versões seguintes via `parent_id`) com contagem e atalho para adicionar.

Atalho de pergunta: no questionário, *Perguntas* → + *adicionar*. O campo questionário é virtual (não é coluna). Sem categoria, entra na primeira categoria por `ordem, id`, ou cria uma categoria **Geral** com ordem 1.

4.1 Publicar

`QuestionarioService::publicar`. Só `rascunho` → `publicado`. Qualquer outro status responde 422 e a ação em lote ignora esse id. Publicar **não** exige perguntas. Liberar sala exige.

4.2 Nova versão

`QuestionarioService::criarNovaVersao`. Só a partir de um **publicado**. Copia, numa transação:

1. o questionário, com `versao + 1`, `status = rascunho`, `parent_id` = original, mesmo dono, escola, nome, descrição, público e flag de compartilhamento;
2. categorias (cor, mensagem, imagem) e subcategorias;
3. perguntas (tipo, texto, ordem, obrigatoriedade, peso, imagem) e opções (descrição, pontuação, ordem, cor, emoji);
4. regras cujo `questionario_id` é o original **ou** cuja `categoria_id` foi copiada. O `questionario_id` da regra só é reapontado quando era o questionário original. A `categoria_id` é remapeada.

O original publicado permanece. Aplicações antigas continuam nele. A versão nova só aparece para o aluno depois de publicada e liberada em outra sala.

A cópia da regra grava rótulo, descrição e faixas. **Não copia `nivel`.** Quem gerar nova versão precisa preencher de novo o nível de atenção, senão o painel trata a faixa como "Sem nível".

Registros com `deleted_at` não entram na cópia.

5. Pontuação e classificação

Implementação: `backend-php/src/Services/ResultadoService.php`. Roda em `POST /aplicacoes/{id}/finalizar`, depois de conferir as obrigatórias. Também é chamado pela importação do legado e pelo seed de respostas fictícias.

Para cada resposta do aluno naquela aplicação:

```
pontos = pontuação da opção, ou 0 se não houver opção (texto livre, ou opção apagada)
total[categoria] += pontos × peso da pergunta
```

O peso nulo conta como 1. Texto livre entra no total como 0: não muda a faixa, mas a resposta fica guardada para o relatório.

Para cada categoria que teve pelo menos uma resposta:

```
SELECT rotulo, descricao, nivel
FROM regras_classificacao
WHERE categoria_id = ? AND min_score <= ? AND max_score >= ? AND deleted_at IS NULL
ORDER BY id
LIMIT 1
```

O nível do aluno no painel é o **pior** entre as categorias daquele resultado: `alto` (3) > `moderado` (2) > `baixo` (1). Sem nível não compete. Quem está prioritário em qualquer categoria conta como prioritário, mesmo que outra categoria esteja adequada. Quem está só em atenção não entra na conta de prioritários.

`nivel`	**Rótulo no painel e no Excel**	**Cor de leitura**
`baixo`	Adequado (sem sinal de risco)	menor gravidade
`moderado`	Atenção (acompanhar)	
`alto`	Prioritário (intervir)	maior gravidade

`nivel`	**Rótulo no painel e no Excel**	**Cor de leitura**
nulo / desconhecido	Sem nível	

Resultados antigos sem `nivel` no JSON são completados, na leitura do painel, pela regra atual da mesma categoria com o mesmo rótulo (`Indicadores::resultados`).

6. Sala

Código em `CodigoSala::gerar`: 6 caracteres do alfabeto `A–Z0–9`, até 40 tentativas, único em `codigo_sala`. Na entrada, o código é posto em maiúsculas e perde tudo que não for letra ou número. A API exige de 4 a 12 caracteres depois disso; o gerador sempre produz 6.

Link: `{PUBLIC_APP_URL}/?sala={CODIGO}`. No app web, `?sala=` na URL vira `codigoSalaPendente` e, depois do login do aluno, entra sozinho na sala.

6.1 Quem pode criar (app da equipe)

`POST /api/v1/staff/salas`:

- questionário **publicado**, da mesma escola, não excluído;
- pesquisador que não é admin da escola: só o próprio questionário ou um compartilhado;
- professor que não é admin: só `alvo_tipo = turma` e só turma de `professor_turma`;
- o questionário precisa ter ao menos uma pergunta ativa.

A resposta traz `codigo`, `link`, questionário, escola, turma, `respondentes` (alunos distintos com alguma resposta) e `concluidos` (linhas em `resultados`).

`POST /staff/salas/{id}/encerrar` põe `status = encerrada`. O aluno deixa de listar e de abrir a sala. Respostas já gravadas permanecem.

`GET /staff/questionarios` lista só **publicados**, no escopo da escola. Pesquisador puro: próprios ou compartilhados.

6.2 Quem entra

`AplicacoesController::paraAluno`: aplicação `ativa`, da escola do aluno, e o alvo inclui o aluno:

- `escola` — qualquer aluno da escola;
- `turma` — `turma_id` igual à turma do aluno;
- `aluno` — `aluno_id` igual ao aluno.

`GET /aplicacoes` devolve, para cada sala visível:

```
{
  "id": 1,
  "questionario": { "id": 10, "nome": "Bem-estar emocional - Demo", "versao": 1 },
  "respondidas": 1,
  "total": 2,
  "concluido": false
}
```

`respondidas` conta linhas em `respostas`. `total` conta perguntas ativas. `concluido` é verdadeiro se já existe `resultados` daquele aluno. Sem pontuação.

`GET /salas/{codigo}` é público (sem token): código, nome do questionário, nome da escola, `alvo_tipo`, `status`. 404 se não houver sala ativa com aquele código.

`POST /aplicacoes/entrar-com-codigo` (com token do aluno):

1. Sala ativa, da escola do aluno e com ele no público → `{ aplicacao_id, questionario_nome, codigo }`.
2. Código inexistente → "Não existe sala com o código ...".
3. Sala encerrada → pede um código novo.
4. Outro cadastro com o **mesmo CPF e a mesma data de nascimento** que se encaixa na sala (outra escola, outra turma ou o aluno indicado) → entra com esse cadastro e devolve `token` + `aluno` novos. O app troca a sessão.
5. Escola diferente, sem outro cadastro compatível → diz o nome das duas escolas.
6. Turma diferente → diz a turma da sala e a turma do cadastro (ou que o cadastro está sem turma).
7. Alvo aluno e não é ele → "foi criada para outro aluno."

7. Fluxo do aluno no app

Estado em `AppController` (`mobile/lib/providers/app_providers.dart`). A navegação não usa rotas nomeadas: `ViseApp` escolhe a tela.

```
Sou aluno
  → CPF + data de nascimento
  → termo (versão 1.0), local se estiver offline
  → lista de questionários + campo de código
       (se a URL tinha ?sala=, entra direto)
  → introdução do avatar (uma vez por aluno + aplicação)
  → categorias ("Caminho")
  → perguntas da categoria
  → Concluir (quando toda pergunta de toda categoria tem resposta local)
  → tela "Obrigado" + envio ao servidor
```

Login com rede chama `QuizRepository.atualizarCache()`: `GET /aplicacoes`, depois categorias e perguntas de cada sala, e grava no Drift (SQLite no Android, IndexedDB na web). Sem rede, o login usa a sessão em cache e o instrumento já baixado.

Imagens `http(s)` são baixadas para o diretório de suporte do app (`instrumentos/`), com nome derivado da URL. Na web o download local não ocorre; a tela usa `Image.network`.

7.1 Cache local

Tabela Drift	Conteúdo
`CachedAplicacao`	id da sala, título (nome do questionário), `concluidaLocalmente`, `sincronizadaServidor`
`CachedCategoria`	chave `(aplicacaoId, id)`, título, mensagem, imagem, ordem
`CachedPergunta`	id, categoria, texto, tipo, obrigatória, imagem, ordem
`CachedOpcao`	id, pergunta, descrição, emoji, ordem. Sem pontuação
`LocalProgress`	chave `(aplicacaoId, perguntaId)`: JSON da resposta e data
`OutboxResposta`	fila de envio: `clientUuid`, payload, status `pendente` / `enviando` / `sincronizada` / `erro`

Salas que o servidor não lista mais (encerradas) saem do cache, **exceto** se ainda houver respostas não sincronizadas.

O menu da tela de categorias pode apagar só as respostas locais daquela sala (`limparRespostas`). Isso não apaga o que já está no servidor.

7.2 Respondendo

Cada toque numa opção, ou o envio de um texto não vazio, grava na hora no aparelho:

```
{
  "client_uuid": "<uuid v4>",
  "pergunta_id": 1,
  "opcao_id": 2,
  "texto": null,
  "responded_at": "2026-10-11T20:15:00.000Z"
}
```

Texto livre é detectado se não há opções, ou se `tipo` contém `text` ou `aberta`. O tipo oficial da API é `texto` ou `multipla_escolha`.

A tela abre na primeira pergunta da categoria ainda sem progresso local. A última pergunta volta para as categorias. Não dá para pular uma pergunta de texto em branco: o botão não grava texto vazio.

Concluir só aparece quando toda categoria tem perguntas e toda pergunta tem uma linha em `LocalProgress`. Isso inclui perguntas **opcionais**. O servidor, na finalização, só cobra as `obrigatoria = 1`. Uma opcional em branco impede o botão no app, mas não impediria um `POST /finalizar` feito direto na API.

A introdução do avatar é marcada em `avatar_intro_<alunoId>_<aplicacaoId>` e não se repete naquela sala.

7.3 Sincronização

`SyncRepository.sincronizar(aplicacaoId)`:

1. Sem conectividade → estado `local`, não envia.
2. Outbox com status diferente de `sincronizada` → `POST /aplicacoes/{id}/respostas/lote` com `{ "respostas": [ ...payloads ] }`.
3. Se a sala está `concluidaLocalmente` e ainda não `sincronizadaServidor` → `POST /aplicacoes/{id}/finalizar`.
4. Sucesso → outbox `sincronizada` e sala marcada como sincronizada no servidor.

O envio também dispara ao recuperar a rede (se houver uma sala aberta) e ao sair da conta do aluno, para não perder o token antes de esvaziar a fila. Falha marca a outbox como `erro`; as respostas continuam no aparelho. A tela final tem o botão **Enviar ao servidor**.

O lote aceita no máximo **500** respostas e é uma transação: ou grava todas, ou nenhuma. Cada item passa pela mesma validação do `POST` unitário (`/respostas`).

`finalizar` devolve `{ "status": "concluido" }` e nada de score. Se faltar obrigatória: 422 `"Ainda existem perguntas obrigatórias sem resposta."` A sala precisa continuar ativa e o aluno no público; sala encerrada no meio do caminho impede finalizar.

8. API do aluno (questionário)

Prefixo `/api/v1`. Token opaco do `POST /login` (CPF + `data_nascimento`). Erros: `{ "detail": "..." }`.

Método	Rota	Auth	Efeito
GET	`/aplicacoes`	aluno	salas ativas do público + progresso
GET	`/salas/{codigo}`	não	cartão público da sala ativa
POST	`/aplicacoes/entrar-com-codigo`	aluno	`{ codigo }` → `aplicacao_id` (e talvez novo token)
GET	`/aplicacoes/{id}/categorias`	aluno	id, nome, ordem, cor, mensagem_avatar, imagem_apoio
GET	`/aplicacoes/{id}/categorias/{categoria}/perguntas`	aluno	perguntas + opções sem pontuação nem peso
POST	`/aplicacoes/{id}/respostas`	aluno	uma resposta → `{ status, id }`
POST	`/aplicacoes/{id}/respostas/lote`	aluno	até 500 → `{ status: sincronizado, quantidade }`
POST	`/aplicacoes/{id}/finalizar`	aluno	calcula resultado → `{ status: concluido }`
GET	`/avatar`	aluno	avatar da escola ou global
POST	`/termos`	aluno	`{ versao, texto_hash }`

Corpo de uma resposta: `pergunta_id` (obrigatório), `opcao_id`, `texto`, `tempo_gasto_ms` (≥ 0), `dispositivo`, `responded_at` (ISO 8601), `client_uuid`.

9. API e telas da equipe

Método	Rota	Efeito
GET	`/staff/questionarios?escola_id=`	publicados no escopo
GET	`/staff/turmas?escola_id=`	turmas; professor só as suas
GET	`/staff/salas?escola_id=`	aplicações, mais recentes primeiro
POST	`/staff/salas`	cria sala ativa com código
POST	`/staff/salas/{id}/encerrar`	encerra
GET/POST	`/staff/cadastros/questionarios` e filhos	o mesmo CRUD do painel, com as mesmas permissões
POST	`/staff/cadastros/questionarios/acoes/publicar`	`{ ids: [...] }`
POST	`/staff/cadastros/questionarios/acoes/nova-versao`	`{ ids: [...] }`
GET	`/staff/painel`	indicadores
GET	`/staff/relatorios/opcoes`	filtros e abas
GET	`/staff/relatorios/exportar`	`.xlsx`

No app, a home da equipe tem **Salas** (criar, copiar link, compartilhar, encerrar). No detalhe de um questionário publicado há o atalho de liberar em sala.

Ações do painel em aplicações: **Encerrar** e **Limpar respostas (demo)**. A limpeza apaga `respostas` e `resultados` da aplicação e só o admin geral executa (`su_only`). Não apaga o questionário nem a sala.

10. Painel e relatórios

`Indicadores` + `FiltrosRelatorio` + `RelatorioService`. Os mesmos números no `/admin` e em `GET /staff/painel`.

Filtros: escola, turma, série, turno, sexo, questionário, aplicação e período (`7`, `30`, `90`, `365`, `tudo`, `personalizado` com `inicio` e `fim` no fuso do app). O painel usa período padrão 30 dias; o Excel, `tudo`. O período dos resultados filtra `resultados.created_at`. Respostas avulsas e a distribuição por pergunta filtram `responded_at`.

10.1 Resumo

Campo	Significado
`alunos`, `turmas`, `escolas`	cadastro no recorte, não só quem respondeu
`participantes`	alunos distintos com resultado no período
`participacao`	participantes / alunos, 0–1, ou nulo se não há alunos
`concluidos`	quantidade de resultados (um aluno em dois questionários conta duas vezes)
`em_andamento`	pares aluno+aplicação com resposta e sem resultado
`respostas`	linhas de resposta no período
`aplicacoes_ativas`	salas `ativa` no recorte (não depende do período)
`prioritarios`	alunos com alguma categoria `alto`
`atencao`	alunos com alguma categoria `moderado` e nenhuma `alto`
`ultima_conclusao`	maior `created_at` dos resultados, em UTC

Também: evolução de conclusões por dia (até 62 dias) ou por mês; classificação por categoria (avaliações, soma, mínimo, máximo, média, faixas com quantidade e percentual); participação por turma (menor primeiro); até 10 aplicações; até 10 alertas (resultados com nível `alto`, com as categorias prioritárias).

Um aluno prioritário não é contado de novo em atenção.

10.2 Excel

`GET /staff/relatorios/exportar`. Abas em `abas` separadas por vírgula. `resumo` entra sempre. Padrão: todas com a terceira coluna `true` abaixo. `anonimizar=1` troca nomes por códigos e tira CPF, matrícula e contatos.

Chave	Aba	No padrão?	Conteúdo
`resumo`	Resumo	sim	filtros, indicadores, faixas
`escolas`	Por escola	sim	participação e níveis
`turmas`	Por turma	sim	participação, pendências, níveis
`classificacao`	Classificação	sim	alunos por faixa de cada categoria
`alunos`	Resultados por aluno	sim	pontuação e classificação por categoria
`perguntas`	Respostas por pergunta	sim	quantidade e % de cada opção; pontuação visível aqui

Chave	Aba	No padrão?	Conteúdo
`pendentes`	Pendentes	não	alunos do público de salas ativas sem resultado; traz telefone e responsável para busca ativa
`respostas`	Respostas detalhadas	não	uma linha por resposta (opção, pontos, texto, tempo). Limite 50.000

Pendentes: até 20.000 linhas. A distribuição por pergunta agrupa `pergunta_id` + `opcao_id` e conta textos não vazios à parte.

Perfis sem permissão de resultado recebem 403.

11. Importação do sistema anterior

`backend-php/src/Install/ImportLegado.php`, pela tela `/install` (com `INSTALL_KEY`) ou `php bin/importar-legado.php backup.sql` (simula) e `--aplicar`. Pode repetir: `legado_map` impede duplicata.

No sistema antigo, cada `categoria_pergunta` **é** um questionário. A importação:

- cria um questionário **publicado**, versão 1, compartilhado, dono = `--pesquisador` ou o primeiro admin geral, descrição "Importado do sistema anterior";
- copia esse questionário para cada escola que o usava (`escola_categoria_pergunta`) ou que tem aluno com resposta naquele instrumento;
- cria **uma** categoria com o mesmo nome;
- traz subcategorias, perguntas e opções;
- pergunta com alguma opção ativa vira `multipla_escolha` obrigatória; sem opções, `texto` não obrigatória; peso `1.00`;
- imagem só migra se a URL já for `http`; arquivo local do servidor antigo é contado como aviso e fica de fora;
- regras: `verde` → nível `baixo` / rótulo Adequado; `amarelo` e `laranja` → `moderado` (Atenção e Atenção elevada); `vermelho` → `alto` / Prioritário. Faixas `min_score`–`max_score` e o texto de interpretação são preservados;
- respostas (objetivas e abertas) vão para uma aplicação **encerrada**, alvo escola, com as datas originais. Se a mesma pergunta foi respondida mais de uma vez, fica a de id maior (mais recente no backup);
- quem completou as obrigatórias ganha resultado calculado pela regra nova.

Questionário sem escola vinculada não é importado e entra no relatório. O backup tem dados pessoais e não deve ir para o repositório.

12. Dados de demonstração

`php bin/install.php --seed` (`Install/Seed.php`), escola INEP `00000001`.

Bem-estar emocional - Demo (compartilhado, público "9º ano", publicado):

- Categoria "Como você tem se sentido?" (`#14B8A6`), mensagem "Não existem respostas certas ou erradas…".
- Múltipla escolha obrigatória: "Com que frequência você se sentiu tranquilo nesta semana?" — Nunca 0 😟, Às vezes 1 😐, Frequentemente 2 🙂.

- Texto opcional: "Conte algo que ajudou você a se sentir bem."
- Faixas: 0 = Sinais de sofrimento (`alto`); 1 = Oscilando (`moderado`); 2 = Tranquilo (`baixo`).

Convívio escolar - Demo (não compartilhado):

- Categoria "Na escola" (`#38BDF8`).
- "Você se sente acolhido(a) na sua turma?" — Pouco 0, Mais ou menos 1, Bastante 2.
- Faixas: Pouco acolhimento (`alto`), Acolhimento parcial (`moderado`), Bem acolhido(a) (`baixo`).

Cada um ganha uma aplicação alvo escola, ativa, com código, início ontem e término em um ano.

`--seed-respostas=80` cria alunos `SIM001`… com respostas fictícias nos dois instrumentos (cerca de 22% sem resposta, uns incompletos, o resto concluído com resultado), para encher painel e Excel. Seed fixo `mt_srand(20260930)`.

Aluno de login: CPF `07593256189`, nascimento `2009-10-21`. Pesquisador: `pesquisador@vise.local` / `password`.

13. O que cada campo influencia de fato

Campo	Efeito real
`status` do questionário	`publicado` é exigido para criar sala pelo app da equipe. A listagem do aluno olha a aplicação, não o status
`publico_alvo`	só texto de cadastro
`compartilhado_na_escola`	visibilidade e permissão de liberar sala para outros pesquisadores
`versao` / `parent_id`	histórico; aplicações apontam para um id concreto
`cor` da categoria e da opção	CMS e API; o app do aluno não pinta com elas
`mensagem_avatar` da categoria	cache local; a fala da introdução vem do avatar
`peso`	multiplica a pontuação no resultado
`obrigatoria`	trava o `finalizar` no servidor
`pontuacao`	some no total da categoria; oculto do aluno; visível no Excel da equipe
`nivel`	painel, alertas, prioritário/atenção. Some na cópia de nova versão
`inicia_em` / `termina_em`	exibição; não bloqueiam o aluno
`status` da aplicação	`ativa` libera; `encerrada` some para o aluno
`alvo_tipo`	quem entra na sala e quem aparece em Pendentes

14. Onde está no código

Assunto	Arquivo
Esquema	`backend-php/database/schema.sql`
Publicar, versão, escola, sala vazia, limpar demo	`backend-php/src/Services/QuestionarioService.php`
Código da sala	`backend-php/src/Services/CodigoSala.php`
Soma e regras	`backend-php/src/Services/ResultadoService.php`
Indicadores	`backend-php/src/Services/Indicadores.php`

Assunto	Arquivo
Filtros	`backend-php/src/Services/FiltrosRelatorio.php`
Excel	`backend-php/src/Services/RelatorioService.php`
Salas e questionários do aluno	`backend-php/src/Api/AplicacoesController.php`
Gravar e finalizar	`backend-php/src/Api/RespostasController.php`
Salas da equipe	`backend-php/src/Api/StaffController.php`
Avatar e termo	`backend-php/src/Api/AlunoExtrasController.php`
CRUD, opções em texto, ações	`backend-php/src/Admin/Resources.php`
Quem vê o quê	`backend-php/src/Admin/Scope.php`
Papéis	`backend-php/src/Roles.php`
Rotas	`backend-php/src/App.php`
Seed	`backend-php/src/Install/Seed.php`
Legado	`backend-php/src/Install/ImportLegado.php`
Cache, resposta local	`mobile/lib/data/repositories/quiz_repository.dart`
Fila e finalizar	`mobile/lib/data/repositories/sync_repository.dart`
Tabelas locais	`mobile/lib/data/local/tables.dart`
Lista, código, categorias, perguntas, fim	`mobile/lib/features/home/` , `categories/` , `questions/` , `finish/`
Testes de ponta a ponta	`backend-php/tests/run.php`